

## EXPERIMENT 10

### IMPLEMENTATION OF 3DES ALGORITHM

#### AIM:

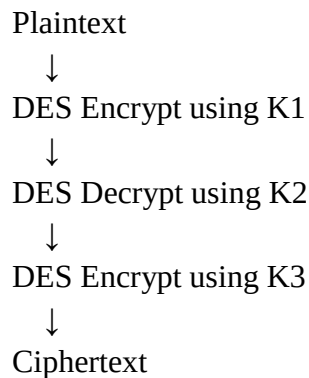
To implement the Triple Data Encryption Standard (3DES) algorithm in C by performing DES encryption/decryption three times using three keys, and to recover the original plaintext through decryption.

#### ALGORITHM:

##### Encryption:

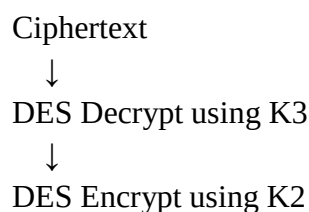
1. Read the 64-bit plaintext and three 64-bit keys K1, K2, and K3.
2. Generate the 16 DES round keys for each of the three keys.
3. Encrypt the plaintext using K1.
4. Decrypt the result using K2.
5. Encrypt the result again using K3.
6. The final result is the 3DES ciphertext.

The encryption process is:



##### Decryption:

1. Decrypt the ciphertext using K3.
2. Encrypt the result using K2.
3. Decrypt the result using K1.
4. The original plaintext is obtained.



↓  
DES Decrypt using K1  
↓  
Plaintext

**PROGRAM:**

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
/* Initial Permutation */
```

```
int IP[64] = {  
    58,50,42,34,26,18,10,2,  
    60,52,44,36,28,20,12,4,  
    62,54,46,38,30,22,14,6,  
    64,56,48,40,32,24,16,8,  
    57,49,41,33,25,17,9,1,  
    59,51,43,35,27,19,11,3,  
    61,53,45,37,29,21,13,5,  
    63,55,47,39,31,23,15,7  
};
```

```
/* Final Permutation */
```

```
int FP[64] = {  
    40,8,48,16,56,24,64,32,  
    39,7,47,15,55,23,63,31,  
    38,6,46,14,54,22,62,30,  
    37,5,45,13,53,21,61,29,  
    36,4,44,12,52,20,60,28,  
    35,3,43,11,51,19,59,27,  
    34,2,42,10,50,18,58,26,  
    33,1,41,9,49,17,57,25
```

```
};
```

```
/* Permuted Choice 1 */
```

```
int PC1[56] = {  
    57,49,41,33,25,17,9,  
    1,58,50,42,34,26,18,  
    10,2,59,51,43,35,27,  
    19,11,3,60,52,44,36,  
    63,55,47,39,31,23,15,  
    7,62,54,46,38,30,22,  
    14,6,61,53,45,37,29,  
    21,13,5,28,20,12,4  
};
```

```
/* Permuted Choice 2 */
```

```
int PC2[48] = {  
    14,17,11,24,1,5,  
    3,28,15,6,21,10,  
    23,19,12,4,26,8,  
    16,7,27,20,13,2,  
    41,52,31,37,47,55,  
    30,40,51,45,33,48,  
    44,49,39,56,34,53,  
    46,42,50,36,29,32  
};
```

```
/* Expansion */
```

```
int E[48] = {  
    32,1,2,3,4,5,
```

```
4,5,6,7,8,9,  
8,9,10,11,12,13,  
12,13,14,15,16,17,  
16,17,18,19,20,21,  
20,21,22,23,24,25,  
24,25,26,27,28,29,  
28,29,30,31,32,1  
};
```

```
/* P Permutation */  
int P[32] = {  
    16,7,20,21,29,12,28,17,  
    1,15,23,26,5,18,31,10,  
    2,8,24,14,32,27,3,9,  
    19,13,30,6,22,11,4,25  
};
```

```
/* Left shifts */  
int SHIFTS[16] = {  
    1,1,2,2,2,2,2,2,  
    1,2,2,2,2,2,2,1  
};
```

```
/* DES S-Boxes */  
int S[8][4][16] = {  
  
    {  
        {14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7},  
        {0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8},
```

{4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0},  
{15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13}  
},

{  
{15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10},  
{3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5},  
{0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15},  
{13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9}  
},

{  
{10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8},  
{13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1},  
{13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7},  
{1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12}  
},

{  
{7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15},  
{13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9},  
{10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4},  
{3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14}  
},

{  
{2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9},  
{14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6},  
{4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14},

```
{11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3}  
},
```

```
{  
{12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11},  
{10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8},  
{9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6},  
{4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13}  
},
```

```
{  
{4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1},  
{13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6},  
{1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2},  
{6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12}  
},
```

```
{  
{13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7},  
{1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2},  
{7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8},  
{2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11}  
}  
};
```

```
/* Permutation function */
```

```
uint64_t permute(uint64_t input, int table[],  
                 int n, int inputBits)
```

```
{
```

```

uint64_t output = 0;

int i;

for(i = 0; i < n; i++)
{
    output <<= 1;
    output |= (input >>
                (inputBits - table[i])) & 1ULL;
}

return output;
}

/* Left rotation */
uint32_t leftRotate28(uint32_t x, int shift)
{
    return ((x << shift) |
            (x >> (28 - shift))) & 0xFFFFFFFF;
}

/* Generate DES keys */
void generateKeys(uint64_t key, uint64_t keys[16])
{
    uint64_t permutedKey;
    uint32_t C, D;
    int i;

    permutedKey =
        permute(key, PC1, 56, 64);

```

```
C = (uint32_t)(permutedKey >> 28)
    & 0xFFFFFFFF;
```

```
D = (uint32_t)permutedKey
    & 0xFFFFFFFF;
```

```
for(i = 0; i < 16; i++)
{
    C = leftRotate28(C, SHIFTS[i]);
    D = leftRotate28(D, SHIFTS[i]);
```

```
uint64_t combined =
    ((uint64_t)C << 28) | D;
```

```
keys[i] =
    permute(combined, PC2, 48, 56);
}
}
```

```
/* Feistel function */
uint32_t feistel(uint32_t R,
    uint64_t roundKey)
{
    uint64_t expanded;
    uint64_t xored;
    uint32_t sboxOutput = 0;
    int i;
```

```

expanded =
    permute((uint64_t)R, E, 48, 32);

xored = expanded ^ roundKey;

for(i = 0; i < 8; i++)
{
    int shift = 42 - (i * 6);

    int sixBits =
        (int)((xored >> shift) & 0x3F);

    int row =
        ((sixBits & 0x20) >> 4)
        | (sixBits & 0x01);

    int col =
        (sixBits >> 1) & 0x0F;

    sboxOutput =
        (sboxOutput << 4)
        | S[i][row][col];
}

return (uint32_t)
    permute(sboxOutput, P, 32, 32);
}

/* DES */

```

```

uint64_t DES(uint64_t block,
              uint64_t keys[16],
              int decrypt)
{
    uint64_t ip;
    uint32_t L, R;
    int i;

    ip = permute(block, IP, 64, 64);

    L = (uint32_t)(ip >> 32);
    R = (uint32_t)ip;

    for(i = 0; i < 16; i++)
    {
        uint32_t oldR = R;

        int keyIndex =
            decrypt ? 15 - i : i;

        R =
            L ^ feistel(R, keys[keyIndex]);

        L = oldR;
    }

    uint64_t combined =
        ((uint64_t)R << 32) | L;

```

```

        return permute(combined, FP, 64, 64);
    }

/* 3DES Encryption: E(K1) -> D(K2) -> E(K3) */
uint64_t TripleDES_Encrypt(
    uint64_t plaintext,
    uint64_t k1[],
    uint64_t k2[],
    uint64_t k3[])
{
    uint64_t x;

    x = DES(plaintext, k1, 0);
    x = DES(x, k2, 1);
    x = DES(x, k3, 0);

    return x;
}

/* 3DES Decryption: D(K3) -> E(K2) -> D(K1) */
uint64_t TripleDES_Decrypt(
    uint64_t ciphertext,
    uint64_t k1[],
    uint64_t k2[],
    uint64_t k3[])
{
    uint64_t x;

    x = DES(ciphertext, k3, 1);

```

```

    x = DES(x, k2, 0);
    x = DES(x, k1, 1);

    return x;
}

int main()
{
    uint64_t plaintext;
    uint64_t key1, key2, key3;
    uint64_t intermediate1;
    uint64_t intermediate2;
    uint64_t ciphertext;
    uint64_t decrypted;

    uint64_t keys1[16];
    uint64_t keys2[16];
    uint64_t keys3[16];

    printf("3DES ENCRYPTION AND DECRYPTION\n");
    printf("-----\n");

    printf("Enter 16-digit hexadecimal plaintext: ");
    scanf("%llx",
        (unsigned long long *)&plaintext);

    printf("Enter 16-digit hexadecimal Key 1: ");
    scanf("%llx",
        (unsigned long long *)&key1);

```

```
printf("Enter 16-digit hexadecimal Key 2: ");  
scanf("%llx",  
      (unsigned long long *)&key2);
```

```
printf("Enter 16-digit hexadecimal Key 3: ");  
scanf("%llx",  
      (unsigned long long *)&key3);
```

```
/* Generate round keys */  
generateKeys(key1, keys1);  
generateKeys(key2, keys2);  
generateKeys(key3, keys3);
```

```
/* First stage: DES Encryption using K1 */  
intermediate1 =  
    DES(plaintext, keys1, 0);
```

```
/* Second stage: DES Decryption using K2 */  
intermediate2 =  
    DES(intermediate1, keys2, 1);
```

```
/* Third stage: DES Encryption using K3 */  
ciphertext =  
    DES(intermediate2, keys3, 0);
```

```
/* Decryption */  
decrypted =  
    TripleDES_Decrypt(  
        ciphertext,  
        keys1,  
        keys2,  
        keys3,  
        1);
```

```

        ciphertext,
        keys1, keys2, keys3);

printf("\nPlaintext      : %016llX",
        (unsigned long long)plaintext);

printf("\nKey 1          : %016llX",
        (unsigned long long)key1);

printf("\nKey 2          : %016llX",
        (unsigned long long)key2);

printf("\nKey 3          : %016llX",
        (unsigned long long)key3);

printf("\nAfter DES with K1 : %016llX",
        (unsigned long long)intermediate1);

printf("\nAfter DES with K2 : %016llX",
        (unsigned long long)intermediate2);

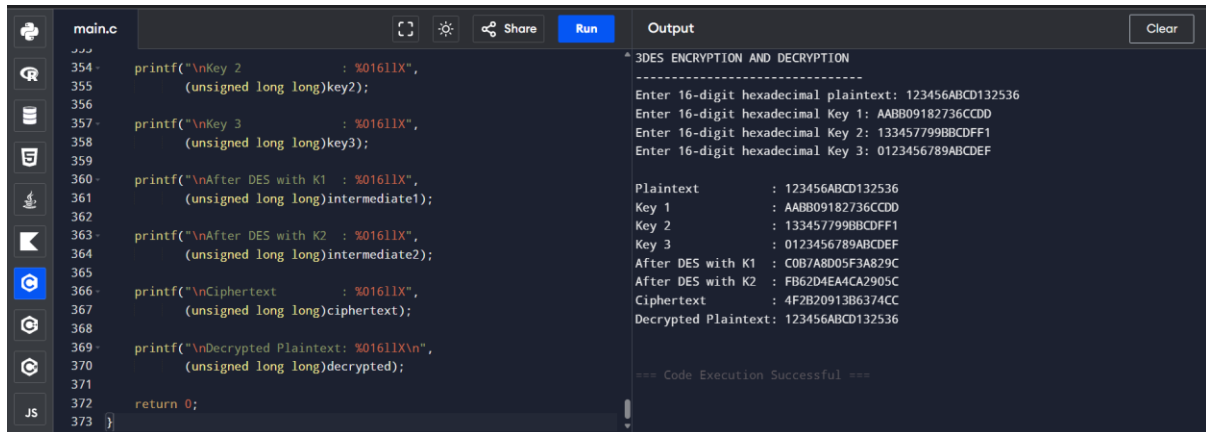
printf("\nCiphertext      : %016llX",
        (unsigned long long)ciphertext);

printf("\nDecrypted Plaintext: %016llX\n",
        (unsigned long long)decrypted);

return 0;
}

```

## OUTPUT:



```
main.c
354 printf("\nKey 2 : %016lX",
355 (unsigned long long)key2);
356
357 printf("\nKey 3 : %016lX",
358 (unsigned long long)key3);
359
360 printf("\nAfter DES with K1 : %016lX",
361 (unsigned long long)intermediate1);
362
363 printf("\nAfter DES with K2 : %016lX",
364 (unsigned long long)intermediate2);
365
366 printf("\nCiphertext : %016lX",
367 (unsigned long long)ciphertext);
368
369 printf("\nDecrypted Plaintext: %016lX\n",
370 (unsigned long long)decrypted);
371
372 return 0;
373 }
```

Output

3DES ENCRYPTION AND DECRYPTION  
-----  
Enter 16-digit hexadecimal plaintext: 123456ABCD132536  
Enter 16-digit hexadecimal Key 1: AAB809182736CCDD  
Enter 16-digit hexadecimal Key 2: 133457799BBCDFF1  
Enter 16-digit hexadecimal Key 3: 0123456789ABCDEF

Plaintext : 123456ABCD132536  
Key 1 : AAB809182736CCDD  
Key 2 : 133457799BBCDFF1  
Key 3 : 0123456789ABCDEF  
After DES with K1 : C0B7A8D05F3A829C  
After DES with K2 : FB62D4EA4CA2905C  
Ciphertext : 4F2B20913B6374CC  
Decrypted Plaintext: 123456ABCD132536

=== Code Execution Successful ===

## RESULT:

Thus, the 3DES algorithm was successfully implemented in C using three different DES keys. The plaintext was encrypted through three DES operations and successfully decrypted to obtain the original plaintext.